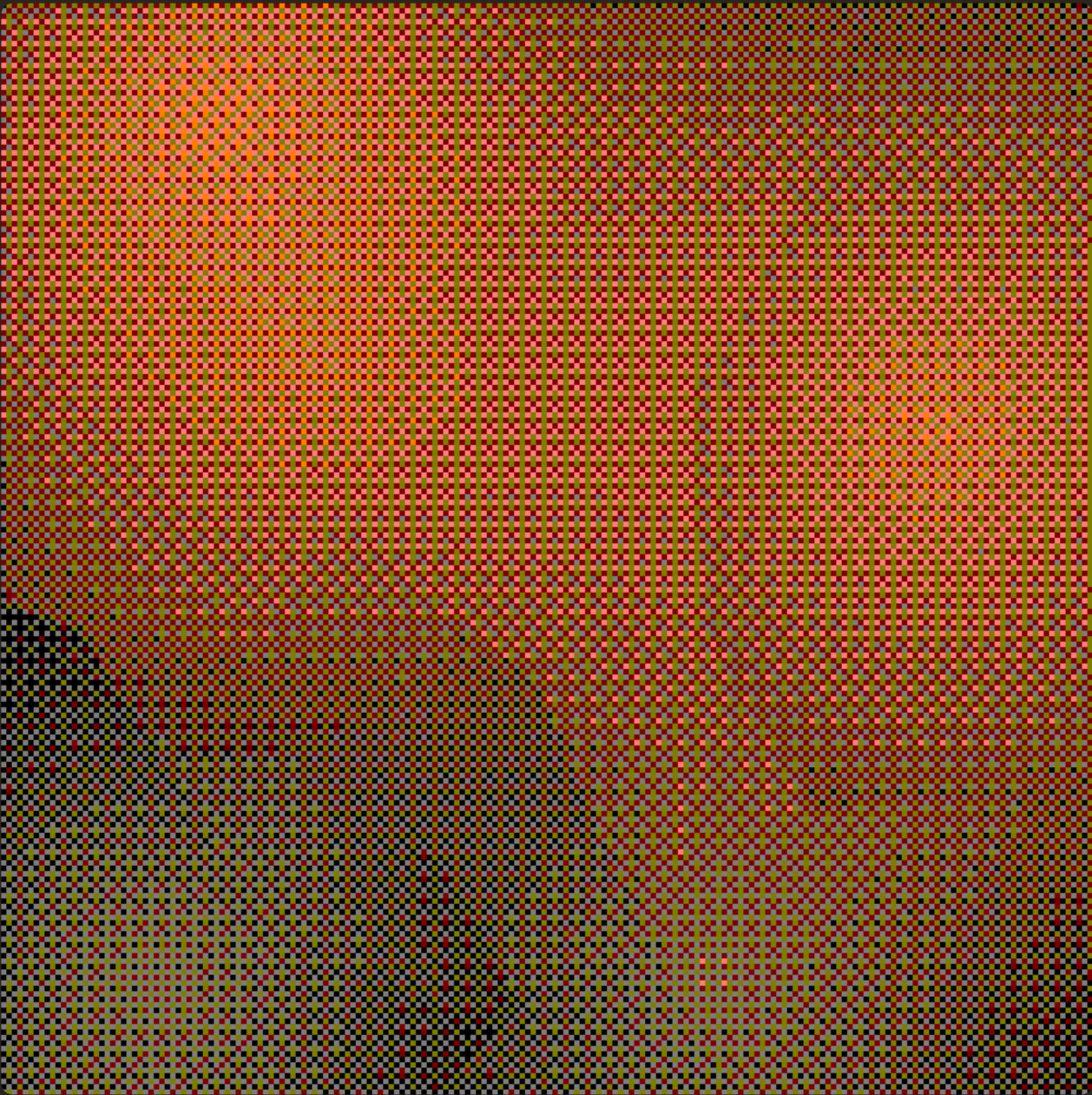


Audit of Midnight Circuits (2)

APRIL 11TH, 2025 • TECHNICAL REPORT



Introduction

On March 31st, 2025, zkSecurity was engaged to audit additional Midnight circuits from Midnight's standard library. The specific codebase was shared via a private GitHub repository (<https://github.com/midnightntwrk/midnight-zk>) at commit `6e1454ef5c5204bd4705a90b3353daabd5f87428` ; this code originally resided in an internal repository and was archived in the new repository prior to publication for transparency and future reference. The audit was conducted between March 31st and April 11th, 2025, by two consultants. Overall, the code was found to be clean, well-organized, and thoughtfully structured. Several observations and findings were reported to the Midnight team, which are detailed in the following sections.

Scope

The following files/directories were the scope of the audit:

- Biguint gadget [src/biguint](#)
- More efficient Poseidon implementation [src/hash/poseidon](#)
- The Parser gadget and Base64 chip [src/json](#)
- Optimization of sgn0 function [src/field/native/native_gadget.rs](#)
- ZkStdLib [src/compact_std_lib.rs](#)
- Utils (Plonk's interface) [src/utlis/plonk_api.rs](#)

Overview

This section provides an overview of the Midnight library and circuits that were within the scope of this audit.

Compact Standard Library (`compact_std_lib`)

The `compact_std_lib.rs` module serves as the main entry point for the Midnight library. It fulfills several critical roles: configuring chips and gadgets, providing public helper functions for circuit operations (e.g., `assign`), and exposing a public API for composing circuits, setting them up, producing proofs, and verifying proofs. Below, we briefly explain these functionalities.

`ZkStdLibArch` defines a configuration for which chips need to be activated in a specific circuit. It contains boolean flags (or an option for SHA256) for enabling `jubjub`, `poseidon`, `sha256`, `secp256k1`, `bls12_381`, and `base64`. In addition, it provides `to_byte` and `from_byte` functions for serialization and deserialization of the configuration.

`ZkStdLib` includes all available chips and gadgets and exposes the ones enabled by the configuration. The `configure` function of `ZkStdLib` sets up all the selected chips and handles the logic for sharing advice and fixed columns across chips.

Beyond chip configuration, `ZkStdLib` also provides helper functions that users commonly need in their circuits. For example, it exposes functions to return the configured chips, as well as wrapped helper methods such as `assert_true` and `assign`. Specifically, it implements:

- `assert_true`
- `assert_false`
- `lower_than`
- `poseidon`
- `hash_to_curve`
- `sha256`

It also implements instructions for the following traits:

- `AssignmentInstructions`
- `PublicInputInstructions`
- `AssertionInstructions`
- `EqualityInstructions`
- `ConversionInstructions`
- `CanonicityInstructions`
- `DecompositionInstructions`
- `ArithInstructions`
- `ControlFlowInstructions`
- `FieldInstructions`
- `RangeCheckInstructions`
- `BinaryInstructions`

Midnight Circuit Structure

The Midnight circuit structure is defined as follows:

```

/// Circuit structure which is used to create any circuit that can be compiled
/// into keys using the ZK standard library.
#[derive(Clone, Debug)]
pub struct MidnightCircuit<'a, R: Relation> {
    relation: &'a R,
    instance: Value<R::Instance>,
    witness: Value<R::Witness>,
    nb_public_inputs: Rc<RefCell<Option<usize>>>,
}

```

The `Relation` trait simplifies circuit development:

```

pub trait Relation: Clone {
    type Instance: Clone;    // Public inputs
    type Witness: Clone;    // Private inputs

    // Convert instance to field elements
    fn format_instance(instance: &Self::Instance) -> Vec<F>;

    // Define circuit logic
    fn circuit(
        &self,
        std_lib: &ZkStdLib,
        layouter: &mut impl Layouter<F>,
        instance: Value<Self::Instance>,
        witness: Value<Self::Witness>,
    ) -> Result<(), Error>;
}

```

Example: Using the Compact Standard Library

Here's a complete example demonstrating how to create and use a circuit with the Compact Standard Library:

```

#[derive(Clone, Default)]
pub struct PoseidonExample;

impl Relation for PoseidonExample {
    type Instance = F;        // Public input: hash output
    type Witness = [F; 3];    // Private input: message to hash

    fn format_instance(instance: &Self::Instance) -> Vec<F> {
        vec![*instance]
    }

    // Circuit implementation
    fn circuit(
        &self,
        std_lib: &ZkStdLib,
        layouter: &mut impl Layouter<F>,
        _instance: Value<Self::Instance>,
        witness: Value<Self::Witness>,
    ) -> Result<(), Error> {
        // Assign witness values
        let assigned_message = std_lib.assign_many(layouter,

```

```

&witness.transpose_array()?;

    // Compute Poseidon hash
    let output = std_lib.poseidon(layouter, &assigned_message?);

    // Mark output as public input
    std_lib.constrain_as_public_input(layouter, &output)
}

// Configure required chips
fn used_chips(&self) -> ZkStdLibArch {
    ZkStdLibArch {
        jubjub: false,
        poseidon: true,    // Only enable Poseidon
        sha256: None,
        secp256k1: false,
        bls12_381: false,
        base64: false,
    }
}

// Usage example
fn main() {
    const K: u32 = 10;
    let srs = filecoin_srs(K);

    // Setup
    let relation = PoseidonExample;
    let vk = compact_std_lib::setup_vk(&srs, &relation);
    let pk = compact_std_lib::setup_pk(&relation, &vk);

    // Generate proof
    let mut rng = ChaCha8Rng::from_entropy();
    let witness: [F; 3] = core::array::from_fn(|_| F::random(&mut rng));
    let instance = <PoseidonChip<F> as HashCPU<F, F>>::hash(&witness);

    let proof = compact_std_lib::prove(&srs, &pk, &relation, &instance, witness, OsRng)
        .expect("\Proof generation should not fail\");

    // Verify proof
    assert!(compact_std_lib::verify:::<PoseidonExample>(
        &srs.verifier_params(),
        &vk,
        &instance,
        &proof
    ).is_ok())
}

```

Parser and Base64 (json)

In the `json` directory, there are implementations for parsing consecutive bytes and for Base64 decoding. The following sections describe how each of these components is implemented.

Parser Gadget Implementation Overview

The `ParserGadget` provides core functionality for fetching sequences of native elements and byte arrays. It offers two main functions:

1. `get_subsequence`

- Extracts a subsequence of native field elements.
- Keeps the index private (as long as the result also remains private).

```
fn get_subsequence(  
    sequence: &[AssignedNative<F>],  
    idx: &AssignedNative<F>,  
    len: usize  
) -> Vec<AssignedNative<F>>
```

2. `fetch_bytes`

- Higher-level byte sequence extraction.
- Uses chunking optimization for efficiency.
- Handles byte-level operations.

```
pub fn fetch_bytes(  
    &self,  
    layouter: &mut impl Layouter<F>,  
    sequence: &[AssignedByte<F>],  
    idx: &AssignedNative<F>,  
    len: usize,  
) -> Result<Vec<AssignedByte<F>>, Error>
```

The implementation of `get_subsequence` first checks that `idx` is less than `sequence.len() - len` to ensure a valid range and prevent overflows. It then creates a vector of `AssignedCell`s of size `len`, assigning and constraining values from the appropriate sequence elements using the `native.select` function.

To efficiently implement `fetch_bytes`, the code first splits the bytes into field-sized groups (`nb_bytes_per_chunk = F::CAPACITY / 8`) and uses two-level indexing:

- `chunk_idx = idx / nb_bytes_per_chunk`
- `fine_search_idx = idx % nb_bytes_per_chunk`

The algorithm follows this structure:

```
Input Bytes -> Chunks -> Selected Chunks -> Output Bytes  
[b0, b1, b2, ...] -> [chunk0, chunk1, ...] -> [selected] ->  
[output bytes]
```

Initially, input bytes are grouped into chunks of size `nb_bytes_per_chunk`. Each chunk is then converted into a field element using `assigned_from_le_bytes`. A dummy chunk is added at the end to handle cross-chunk boundary cases. The index is decomposed and constrained via:

```
idx = chunk_idx * nb_bytes_per_chunk + fine_search_idx
```

Additionally, `fine_search_idx` is range-checked to ensure it lies within `[0, nb_bytes_per_chunk)`. The number of required chunks is computed as:

```
len_for_chunks = min(nb_chunks, 1 + len.div_ceil(nb_bytes_per_chunk))
```

Selected chunks are extracted using `get_subsequence`, with the dummy chunk ensuring correct behavior at boundaries. Note that the `chunk_idx` is range-checked through the `get_subsequence` function call. The selected chunks are then decomposed back into bytes using `assigned_to_le_bytes`, and the final bytes are selected via another `get_subsequence` using `fine_search_idx`.

Base64 Chip Implementation Overview

The [Base64 chip](#) implements Base64 decoding, supporting both standard Base64 and Base64URL encodings. The implementation uses a lookup table that maps pairs of ASCII characters to their Base64-encoded values. The table optimizes lookups by grouping two characters at a time:

```
t_char (16-bit)    t_val (12-bit)
0x4141 ('AA')      0x0000 (0,0)
0x4142 ('AB')      0x0001 (0,1)
...
0x2F2F ('//')      0x0FFF (63,63)
```

The core function of the chip is `base64_to_val_chunk`. This function receives 4 characters (`AssignedByte`) and returns 4 decoded `AssignedNative` values. The circuit uses four advice columns organized as:

LOOKUP_SEL	COL_1	COL_2	COL_3	COL_4
1	A	B	0x01	0x01
1	C	D	0x02	0x03

In `base64_to_val_chunk`, the input bytes are assigned into the first two advice columns (`col_1` and `col_2`). They are decoded and assigned into the next two columns (`col_3` and `col_4`), and the `lookup_sel` selector is enabled for both rows. The `Base64` lookup gate:

- Combines columns as $(col_1 \ll 8) \mid col_2$ and $(col_3 \ll 6) \mid col_4$.
- Performs a lookup in the precomputed table to validate that the pair exists.

This ensures that the combined 12-bit decoded value corresponds to the input 16-bit ASCII pair. However, it does **not** enforce that `col_3` individually decodes `col_1` or `col_4` decodes `col_2` independently (see finding “Incorrect Range Assumptions in Base64 Decoding”). Nevertheless, in the current implementation, no soundness error arises, as the decoded values are immediately processed via `val_to_ascii_chunk`:

```
let terms = vec![
  (F::from(1u64 << (3 * 6)), b64_input[0].clone()),
  (F::from(1u64 << (2 * 6)), b64_input[1].clone()),
  (F::from(1u64 << 6), b64_input[2].clone()),
  (F::ONE, b64_input[3].clone()),
];
```

The chip also implements the following helper functions:

- `url_to_standard`: Converts Base64URL encoding to standard Base64 by replacing '-' with '+' and '_' with '/' using selection gadgets.
- `pad`: Fills an incomplete chunk with `ALT_PAD` ('A') characters to ensure a 4-byte chunk.
- `process_padded_chunk`: Processes Base64 chunks containing '=' padding characters by replacing them with `ALT_PAD` for proper decoding.
- `from_base64`: Decodes a Base64 string into ASCII output, handling chunking and padding.
- `from_base64url`: Converts Base64URL strings to standard Base64, then decodes them via `from_base64`.

Poseidon (Optimized Implementation)

The Midnight implementation of Poseidon has been updated to incorporate a major optimization inspired by [ePrint 2022/462](#). For a thorough explanation of the older implementation and how the Poseidon chip is constructed and can be used, we refer the interested reader to our previous [report](#). In this section, we are going to describe the full round and mainly the partial rounds implementation that have been optimized. In summary, the optimization process multiple partial rounds in a single batch that results in a single row in the plonkish table. This is achieved by:

- **Shifted Rounds**: A change in round structure to simplify constraint expressions.

- **Round Skipping:** A technique that combines multiple partial rounds into a single constraint.

These changes significantly reduce the number of constraints in the circuit while preserving the security guarantees of the original Poseidon construction.

Shifted Rounds: Simplified Constraint Structure

The original Poseidon permutation applies the round constants *before* the S-box and MDS multiplication:

```
state = input
round 0: state = MDS * Sbox(state + RC[0])
round 1: state = MDS * Sbox(state + RC[1])
...
round n: state = MDS * Sbox(state + RC[n])
output = state
```

The optimized implementation instead applies the round constants *after* the MDS and S-box layers, and shifts them forward by one round. The first round constant is added to the input directly:

```
state = input + RC[0]
round 0: state = MDS * Sbox(state) + RC[1]
round 1: state = MDS * Sbox(state) + RC[2]
...
round n-1: state = MDS * Sbox(state) + RC[n]
round n : state = MDS * Sbox(state) output = state
```

This structure mainly simplifies the polynomial identities enforced in circuit constraints. Note that this optimization is applied in the **partial** rounds.

Round Skipping: Batched Partial Rounds

In the standard Poseidon construction, partial rounds apply the S-box to only a single element in the state (typically the first or last). In the optimized implementation, multiple partial rounds are combined into a single constraint using a *skipping* mechanism.

Configuration

Two versions are supported:

```
pub(crate) const NB_SKIPS_CIRCUIT: usize = 5; // For circuit
pub(crate) const NB_SKIPS_CPU:   usize = 2;  // For native (CPU)
```

The constant `NB_SKIPS_CIRCUIT` controls how many partial rounds are batched together in the circuit, reducing the number of constraints per round. Note that in the paper, it mentions that `2` is the ideal number for CPU, but for constraints there are different trade-offs between different options

Round Representation

The round evaluation process is abstracted through the precomputed structure:

```
pub struct PreComputedRoundCircuit<F: PoseidonField> {
    pub(crate) round_constants: RoundConstantsCircuit<F>,
    pub(crate) partial_round_id: RoundId<F>,
}
```

These store optimized constants and polynomial identities used in the round-skipping gates. A helper function evaluates these batched rounds:

```
fn eval<const NB_SKIPS: usize>(
    &self,
    round_constants: &[F], // Length WIDTH + NB_SKIPS
    instances: &mut [F],   // Current state
) -> [F; NB_SKIPS]       // Intermediate results
```

This design enables compact representation of multiple partial rounds as a single linear combination.

Circuit Gates

The optimized Poseidon circuit defines two custom gates. Before explaining those gates let's focus on the `linear_layer` function.

The `linear_layer` Function

The `linear_layer` function implements the core operation of the linear mixing step in the Poseidon permutation:

```
fn linear_layer<F: PoseidonField>(
    inputs: &[Expression<F>], // Length WIDTH
    outputs: &[Expression<F>], // Length WIDTH
    constants: &mut [Expression<F>], // Length WIDTH
) {
    for i in 0..WIDTH {
        constants[i] = constants[i].clone() - outputs[i].clone();
        for j in 0..WIDTH {
            constants[i] =
                constants[i].clone() + Expression::Constant(F::MDS[i][j]) *
                inputs[j].clone();
        }
    }
}
```

Purpose:

- Compute the identity of the linear layer for the Poseidon round.
- This corresponds to MDS matrix multiplication **plus** addition of round constants.

Inputs:

- `inputs` : Current state before MDS mixing.
- `outputs` : Expected state after S-box and MDS mixing.
- `constants` : Mutable slice initialized with round constants.

Operation:

For each output position `i`, the function:

1. Subtracts the expected output from the corresponding constant:

$$constants[i] = constants[i] - outputs[i]$$
2. Adds the result of the MDS linear transformation applied to `inputs`:

$$constants[i] += \sum_{j=0}^{WIDTH-1} MDS[i][j] \times inputs[j]$$

Thus, after running `linear_layer`, the constraints are ready to be enforced as polynomial equalities:

$$constants[i] = 0 \quad \text{for each } i \in [0, WIDTH - 1]$$

Implementation Notes:

- The function mutates `constants` to avoid allocating new vectors.
- It expects `constants` to be pre-populated with round constants.

The `linear_layer` is used in both full and partial round gates to validate the correctness of the MDS mixing and round constants application.

1. Full Round Gate

This gate enforces the usual round behavior where all elements are processed through the S-box and MDS matrix.

```
meta.create_gate("full_round_gate", |meta| {
  let mut inputs = ...;
  let outputs = ...;
  let mut identities = ...;

  for x in inputs.iter_mut() {
    *x = x.clone().square().square() * x.clone(); // x^5
  }
})
```

```
linear_layer(&inputs, &outputs, &mut identities);
});
```

Below is the visual representation of this gate.

ROW	Q_FULL_ROUND	STATE[0]	STATE[1]	STATE[2]	RC[0]	RC[1]	RC[2]
i	1	s0	s1	s2	c0	c1	c2
i+1	-	s0'	s1'	s2'	-	-	-

Each output (j in $0..2$) is constrained as:

$$s'_j = MDS[j][0] * (s_0^5) + MDS[j][1] * (s_1^5) + MDS[j][2] * (s_2^5) + c_j$$

What it enforces:

- S-box (x^5) applied to all elements.
- Linear layer with MDS matrix multiplication.
- Correct application of round constants.
- Result matches the next state row.

2. Partial Round Gate (with Skipping)

This gate combines $1 + \text{NB_SKIPS_CIRCUIT}$ partial rounds into a single constraint:

```
meta.create_gate("partial_round_gate", |meta| {
  let inputs = ...;
  let outputs = ...;
  let round_constants = ...;

  let ids = ids.to_expression(&inputs);

  let output_lin_ids = (0..WIDTH - 1).map(|i| round_constants[i] - outputs[i] +
ids[i]);

  let input_pow_ids = (WIDTH - 1..WIDTH + NB_SKIPS_CIRCUIT - 1)
    .map(|i| round_constants[i] - inputs[i + 1] + ids[i]);

  let output_pow_id =
    round_constants[WIDTH + NB_SKIPS_CIRCUIT - 1] - outputs[WIDTH - 1] + ids[WIDTH +
NB_SKIPS_CIRCUIT - 1];
});
```

This results in three categories of constraints:

- **Linear output constraints** for state elements that do not pass through the S-box.
- **Intermediate S-box constraints** for skipped rounds.

- **Final S-box constraint** for the last state element after all skipped rounds.

Below is the visual representation of the constraints

ROW	Q_PARTIAL_ROUND	STATE[0]	STATE[1]	STATE[2]	AUX[0..4]	RC[0..7]
i	1	s0	s1	s2	a0 a1 a2 a3 a4	c0 c1 c2 c3 c4 c5 c6 c7
i+1	-	s0'	s1'	s2'	-	-

Constraints:

1. Linear Output (state[0], state[1]):

$$s0' = \text{linear_combination}(\dots) + c0$$

$$s1' = \text{linear_combination}(\dots) + c1$$

2. S-box Intermediate Values (aux[0..4]):

$$a_k = (a_k)^5 + c_{2+k}$$

3. Final Output (state[2]):

$$s2' = (a_4)^5 + c7$$

Only `state[2]` passes through the S-box in each of the skipped rounds. The auxiliary columns store intermediate S-box values, and all constants are precomputed to match the expected outputs.

Sgn0

The `sgn0` function extracts the least significant bit of a field element: by which is meant the least significant bit of the canonical representation of the field element; the unique integer $e \in \mathbb{N}_{<q}$ st. $e \equiv x \pmod{q}$. The implementation is as follows and we explain its correctness below:

```
fn sgn0(
    &self,
    layouter: &mut impl Layouter<F>,
    x: &AssignedNative<F>,
) -> Result<AssignedBit<F>, Error> {
    // We decompose x as msb || mid_limb || lsb.

    let n = F::NUM_BITS as u64;
    let msb_factor = big_to_fe::<F>(BigUint::one() << (n - 1));
```

```

let x_val = x.value().copied().map(fe_to_big);
let lsb_val = x_val.clone().map(|x| Bit(x.bit(0)));
let msb_val = x_val.clone().map(|x| Bit(x.bit(n - 1)));
let mid_val = x_val.map(|x| {
    let mut x = x;
    x.set_bit(n - 1, false);
    big_to_fe::<F>(x >> 1)
});

let x_lsb: AssignedBit<F> = self.assign(layouter, lsb_val)?;
let x_msb: AssignedBit<F> = self.assign(layouter, msb_val)?;
let x_mid = self.core_decomposition_chip.assign_less_than_pow2(
    layouter,
    mid_val,
    n as usize - 2,
)?;

let must_be_x = self.linear_combination(
    layouter,
    &[
        (F::ONE, x_lsb.clone().into()),
        (F::from(2), x_mid.clone()),
        (msb_factor, x_msb.clone().into()),
    ],
    F::ZERO,
)?;

self.assert_equal(layouter, x, &must_be_x)?;

// At this point we have that (x_msb, x_mid, x_lsb) is a valid split of x.
// We want to make sure it is canonical before we can output x_lsb.
// Let (1, p_mid, 1) be the representation of the native modulus.
// For canonicity, we will check that one of the following is true:
// (1) x_msb = 1 => x_mid <= p_mid
// (2) x_mid = p_mid => x_lsb = 0
let p = modulus::<F>();
debug_assert!(p.bit(0));
debug_assert!(p.bit(n - 1));
let p_mid: BigUint = (p - fe_to_big(msb_factor)) >> 1;

// Check condition (1)
let zero = self.assign_fixed(layouter, F::ZERO)?;
let aux = self.select(layouter, &x_msb, &x_mid, &zero)?;
self.assert_lower_than_fixed(layouter, &aux, &(p_mid.clone() + BigUint::one()))?;

// Check condition (2)
let same_mid = self.is_equal_to_fixed(layouter, &x_mid, big_to_fe(p_mid))?;
let same_mid_and_lsb_set = self.and(layouter, &[same_mid, x_lsb.clone()]);
self.assert_equal_to_fixed(layouter, &same_mid_and_lsb_set, Bit(false))?;

Ok(x_lsb)
}

```

First the value $x \in \mathbb{F}_q$ is decomposed into (range checked) parts:

- $x_{\text{lsb}} \in \{0, 1\}$

- $x_{\text{mid}} \in [0, \dots, 2^{n-1})$
- $x_{\text{msb}} \in \{0, 1\}$

Such that:

- $x = x_{\text{msb}} \cdot 2^{n-1} + x_{\text{mid}} \cdot 2 + x_{\text{lsb}}$

Since this representation is not unique, i.e. $q < 2^n - 1$, we need to ensure that the decomposition is the canonical one: corresponding to the integer $e \in \mathbb{N}_{<q}$ described at the beginning of this section. As indicated by the comment in the code, this is done by enforcing the following two conditions:

- $x_{\text{msb}} = 1 \implies x_{\text{mid}} \leq p_{\text{mid}}$
- $x_{\text{mid}} = p_{\text{mid}} \implies x_{\text{lsb}} = 0$

The first condition covers most of the non-canonical decompositions, namely any non-canonical decomposition must have $x_{\text{msb}} = 1$, matching the most significant bit of q , which then implies that for the representative to be larger than q , the $x_{\text{mid}} \parallel x_{\text{lsb}}$ must be greater than $p_{\text{mid}} \parallel 1$. The one condition not handled by the first requirement is exactly when $x_{\text{mid}} = p_{\text{mid}}$ and $x_{\text{lsb}} = 1$. Corresponding to a decomposition of $0 \in \mathbb{F}_q$ as $q \in \mathbb{Z}$. The last condition serves to account for this case, to enforce it is rewritten by replacing the implication with a disjunction and a negation, then applying DeMorgan's law:

$$x_{\text{mid}} = p_{\text{mid}} \implies x_{\text{lsb}} = 0$$

Rewrite the implication as a disjunction:

$$x_{\text{mid}} \neq p_{\text{mid}} \vee x_{\text{lsb}} = 1$$

And by De Morgan's law:

$$\neg(x_{\text{mid}} = p_{\text{mid}} \wedge x_{\text{lsb}} = 1)$$

BigUInt

In addition to the foreign field emulation, it also has the `biguint` module to emulate arbitrary large integers. It is an useful module for the use cases like RSA verification.

Similarly, it has a normalization process to correct the overflowed limbs. The process is relatively simpler than the one for foreign field emulation.

Limb Representation

The module `biguint` can represent an arbitrary large integer represented by decomposed limbs. The default bound for each limb is $0 \leq x_i < B$, where B has 96 bits.

A `biguint` can be decomposed and encoded into a circuit using the `assign_bounded` function, which takes the `nb_bits` to decide how many limbs to create and the bounds for each limb.

$$X = \sum_{i=0}^{n-1} x_i \cdot B^i$$

The bounds for the limbs are the same as B , except for the most significant limb.

$$x_i = \begin{cases} 0 \leq x_i < B, 0 \leq i < n - 1 \\ 0 \leq x_i < B_{\text{msl}}, i = n - 1 \end{cases}$$

Code for the limb bounds:

```
let mut limb_size_bounds = vec![LOG2_BASE; nb_limbs - 1];
limb_size_bounds.push((nb_bits - 1).rem(LOG2_BASE) + 1); // msl bound
```

Code for range check the limbs:

```
let limbs = value
    .map(|x| big_to_limbs(nb_limbs as u32, &(BigUint::one() << LOG2_BASE), &x))
    .transpose_vec(nb_limbs)
    .into_iter()
    .zip(limb_size_bounds.iter())
    .map(|(limb_value, size_bound)| {
        self.native_gadget.assign_lower_than_fixed(
            layouter,
            limb_value.map(big_to_fe::<F>),
            &(BigUint::one() << *size_bound),
        )
    })
    .collect::<Result<Vec<_>, Error>>()?;
```

During operations, such as Addition or Multiplication, the value can overflow its limb's bound. In `biguint` module, there is also a normalization process to correct these limb overflows. This procedure relies on the bound tracker at the compile time to determine whether or not to normalize a `biguint`.

Bound Adjustment

The basic operations Addition, Subtraction and Multiplication also calculate the limb bounds for the resulted `biguint`.

Addition and Subtraction Operations

In addition operation, when two limbs sum up, the resulted value can overflow the upper bound of them.

$$z_i = x_i + y_i$$

So it needs to add one bit to the upper bound in order to represent the carry bit during normalization.

$$b_{z_i} = 1 + \max(b_{x_i}, b_{y_i})$$

Where b represents the bound for the limbs.

```
limbs.push(self.native_gadget.add(layouter, &x.limbs[i], &y.limbs[i]));
limb_size_bounds.push(bound_of_addition(
    x.limb_size_bounds[i],
    y.limb_size_bounds[i],
));
```

On the other hand, the subtraction simply picks the bounds of the left hand side, as it assumes the LHS is always greater than or equal to the RHS.

$$b_{z_i} = b_{x_i}$$

```
let res = self.assign_bounded(layouter, res_value, x.nb_bits());
let z = self.add(layouter, &res, y); // (x - y) + y
self.assert_equal(layouter, x, &z);
Ok(res)
```

Multiplication Operation

Supposed there are two `biguint`, X and Y , their multiplication can be represented as Z .

$$X = \sum_{i=0}^{n-1} x_i \cdot B^i, \quad Y = \sum_{j=0}^{m-1} y_j \cdot B^j$$

$$X \cdot Y = \sum_{i=0}^{n-1} \sum_{j=0}^{m-1} x_i \cdot y_j \cdot B^{i+j}$$

$$Z = X \cdot Y = \sum_{k=0}^{n+m-2} z_k \cdot B^k$$

Let Z_k represents the resulted limbs:

$$z_k = \sum_{i+j=k} x_i \cdot y_j$$

As the bound for the z_k is $b_{z_k} = b_{x_i} + b_{y_j}$, and there are accumulations to z_k , thus the bound should be accumulated as:

$$b_{z_k} = 1 + \max(z_k, b_{x_i} + b_{y_i})$$

```
for i in 0..x.limbs.len() {
    for j in 0..y.limbs.len() {
        let p = native_gadget.mul(layouter, &x.limbs[i], &y.limbs[j], None)?;
        let p_bound = x.limb_size_bounds[i] + y.limb_size_bounds[j];
        limbs[i + j] = native_gadget.add(layouter, &limbs[i + j], &p)?;
        limb_size_bounds[i + j] = bound_of_addition(limb_size_bounds[i + j], p_bound);
    }
}
```

```
}
}
```

Normalization Process

Similar to the normalization in the foreign field module, it also need to run normalization procedure to convert the limbs with overflow values to fall within the expected bound.

The limb overflow can happen during the basic operations addition and multiplication. These operations also update each bound of the limbs in order to keep track of whether it needs to be normalized and how to normalize.

Before starting the normalization procedure, the `normalize()` check if the limb bounds representing a `biguint` is within the default bound, 96 bits.

```
pub fn is_normalized(&self) -> bool {
    self.limb_size_bounds
        .iter()
        .all(|bound| *bound <= LOG2_BASE)
}
```

Note that this seems not strictly aligning with the actual bound of the most significant limb. Since the operations, involving updating the bounds, will update bounds for all the limbs. Thus, a single update to the limb bounds can lead the `biguint` to be hint by the normalization process eventually. So far, it is fine this `is_normalized` check doesn't strictly align with the actual bound of the most significant limb.

First, it calculates the number of limbs based on the number of bits, which are derived from the max limb bounds.

```
let nb_limbs_output = x.nb_bits().div_ceil(LOG2_BASE) as usize;
```

Second, it goes through each potential overflow limb to calculate the `payload` of the limb. Note that the `carry0` is 0.

$$\text{payload}_i = x_i + \text{carry}_i$$

```
let payload = native_gadget.add(layouter, &carry, &x.limbs[i])?;
let payload_bound = bound_of_addition(carry_size_bound, x.limb_size_bounds[i]);
```

Third, it normalizes the limb x_i :

$$x'_i = \text{payload}_i \bmod B$$

Before proceeding to the next limb, it calculates the `carryi+1`.

$$\text{carry}_{i+1} = \left\lfloor \frac{\text{payload}_i}{B} \right\rfloor$$

```
let (q, limb) = self.div_rem_native_by_base(layouter, &payload, payload_bound)?;
// Prepare the carry and its bound for the next iteration.
carry_size_bound = max(payload_bound, LOG2_BASE) - LOG2_BASE;
carry = q;
```

The `div_rem_native_by_base` function is used to calculate the quotient `q` and the remainder `limb` of the division by the base B .

$$\text{payload}_i = q \cdot B + \text{limb}_{\text{normalized}}$$

It relies on the `payload_bound` to range check the `q`, the carry_{i+1} . After all limbs are processed, it ensures the final `carry` is 0.

Plonk API Utils

The `Plonk API Utils` ([src/utls/plonk_api.rs](#)) provide a set of utilities and helper functions for set up, proving, and verifying PLONK circuits using Midnight's Halo2 fork.

The main functionalities include:

- **Proving and Verification Utilities:** The `plonk_api!` macro generates structs that expose standardized methods for setting up verifying keys (`setup_vk`), proving keys (`setup_pk`), creating proofs (`prove`), and verifying proofs (`verify`). These abstractions can be instantiated for several curves.
- **Verification Key Management:** The `check_vk` function ensures that verification keys for circuits remain consistent across code changes. It compares the computed VK hash against stored values, enforcing versioning policies:
 - If a VK is missing, a minor version bump is required.
 - If a VK differs, a breaking change must be recorded.
- **SRS Handling for Filecoin's Trusted Setup:** The `filecoin_srs` function loads a Structured Reference String (SRS) compatible with BLS12-381 Filecoin trusted setup. If necessary, it can download and parse raw SRS data from specified sources and downsize the parameters to the required circuit size.

Findings

Below are listed the findings found during the engagement. High severity findings can be seen as so-called "priority 0" issues that need fixing (potentially urgently). Medium severity findings are most often serious findings that have less impact (or are harder to exploit) than

high-severity findings. Low severity findings are most often exploitable in contrived scenarios, if at all, but still warrant reflection. Findings marked as informational are general comments that did not fit any of the other criteria.

ID	COMPONENT	NAME	RISK
#00	compact_std_lib.rs	Deserializing an Architecture Description can cause a crash	Medium
#01	json/base64_chip.rs	Incorrect Range Assumptions in Base64 Decoding	Low
#02	compact_std_lib.rs	Batch Verification subject to Truncation	Low
#03	poseidon	State Management and Implicit Assumptions in Poseidon Implementation	Low
#04	json	Miscellaneous Remarks on JSON	Informational
#05	poseidon	Miscellaneous Remarks on Poseidon	Informational
#06	poseidon	Poseidon Parameters Generated by a Deprecated Script	Informational
#07	compact_std_lib	Miscellaneous Remarks on ZkStdLib	Informational

#00 - Deserializing an Architecture Description can cause a crash

Severity: Medium **Location:** compact_std_lib.rs

Description

ZkStdLib supports multiple “architectures”, combinations of enabled gadgets, affecting the number of columns during configuration. To enable the verifier to know which architecture is being used for a circuit (a circuit might be proven with different architectures) the flags indicating which gadgets are enabled, stored in `ZkStdLibArch` are packed into a single `u8`. This is then serialized by the verifier:

```
impl ZkStdLibArch {
    ..
    fn from_byte(byte: u8) -> Self {
        ZkStdLibArch {
            jubjub: byte & 1 != 0,
            poseidon: byte & 2 != 0,
            sha256: match (byte >> 2) & 0b11 {
                0 => None,
                1 => Some(ShaTableSize::Table11),
                2 => Some(ShaTableSize::Table16),
                _ => unreachable!(),
            },
            secp256k1: byte & 16 != 0,
            bls12_381: byte & 32 != 0,
            base64: byte & 64 != 0,
        }
    }
}
```

The problem is that when `(byte >> 2) & 0b11` is equal to 3, the `match` expression will panic: the `unreachable!()` branch is reachable. We suggest implementing `TryFrom<u8>` for `ZkStdLibArch` and to handle this case gracefully.

Impact

The immediate impact is that the verifier will crash when deserializing the architecture description. This is a potential liveness issue in the wider context of midnight, since validators deserialize untrusted verification keys.

Client Response

Midnight implemented the `TryFrom<u8>` trait for `ZkStdLibArch` and handled the case where `(byte >> 2) & 0b11` is equal to 3 by returning an error:

```
impl TryInto<ZkStdLibArch> for u8 {
    type Error = io::Error;

    fn try_into(self) -> Result<ZkStdLibArch, Self::Error> {
```

```

Ok(ZkStdLibArch {
  jubjub: self & 1 != 0,
  poseidon: self & 2 != 0,
  sha256: match (self >> 2) & 0b11 {
    0 => None,
    1 => Some(ShaTableSize::Table11),
    2 => Some(ShaTableSize::Table16),
    _ => return Err(io::Error::from(io::ErrorKind::InvalidInput)),
  },
  secp256k1: self & 16 != 0,
  bls12_381: self & 32 != 0,
  base64: self & 64 != 0,
})
}
}

```

#01 - Incorrect Range Assumptions in Base64 Decoding

Severity: Low **Location:** json/base64_chip.rs

Description

The base64 decoding gadget is centered around a table, mapping two digit Base64 ASCII string to its decoded value.

TWO BASE64 DIGITS	DECODED VALUE
'AA'	0
'AB'	1
'AC'	2
...	...
'/'	$2^{12} - 1$

The lookups in this table are implemented in-circuit by the `Base64 lookup` gate:

```
fn configure(
    meta: &mut halo2_proofs::plonk::ConstraintSystem<F>,
    shared_resources: &Self::SharedResources,
) -> Self::Config {
    ...

    meta.lookup("Base64 lookup", |meta| {
        let s = meta.query_selector(lookup_sel);

        // characters = first_col * 8 + second_col
        let col_1 = meta.query_advice(advice_cols[0], Rotation::cur());
        let col_2 = meta.query_advice(advice_cols[1], Rotation::cur());
        let characters = col_1 * Expression::Constant(F::from(1 << 8)) + col_2;

        // values = third_col * 6 + fourth_col
        let col_3 = meta.query_advice(advice_cols[2], Rotation::cur());
        let col_4 = meta.query_advice(advice_cols[3], Rotation::cur());
        let values = col_3 * Expression::Constant(F::from(1 << 6)) + col_4;

        // default value for the deactivated lookup
        let default_char =
            Expression::Constant(F::from(super::table::two_entry_default()));

        vec![
            (
                s.clone() * characters
                + (Expression::Constant(F::ONE) - s.clone()) * default_char,
                t_char,
            )
        ]
    })
```

```

    ),
    (s.clone() * values, t_val),
  ]
});

...
}

```

Which performs a lookup in the fixed table:

- The gate allocates four cells:
 - col_1 and col_2 for two Base64 characters.
 - col_3 and col_4 for the resulting 6-bit values.
- If $lookup_{sel} = 1$, it then checks:

$$(col_1 \cdot 2^8 + col_2, col_3 \cdot 2^6 + col_4) \in \text{Base64Table}$$

Since the least common multiple of:

- 8 : the number of bits in a byte.
- 6 : the number of bits required to encode the range of a single Base64 character.

Is 24, we decode the Base64 characters in groups of 24 bits, corresponding to 4 base64 characters and 3 bytes. Therefore, this gate is invoked twice during synthesis, to handle two times two Base64 characters, this is done by `base64_to_val_chunk` :

```

/// Receives 4 ascii characters as [AssignedByte] representing a base64
/// string. Returns a vector of 4 [AssignedNative] with each symbol
/// values. These values are guaranteed to be in the range [0, 2^6).
fn base64_to_val_chunk(
  &self,
  layouter: &mut impl Layouter<F>,
  b64_input: &B64Chunk<F>,
) -> Result<[AssignedNative<F>; 4], Error> {
  let advice_cols = self.config.advice_cols;
  // |-----|-----|-----|-----|
  // | A     | B     | 0x00 | 0x01 |
  // |-----|-----|-----|-----|
  // | C     | D     | 0x02 | 0x03 |
  // |-----|-----|-----|-----|
  let decoded = layouter.assign_region(
    || "Base64 chunk",
    |mut region| {
      let decoded_outs: Vec<_> = b64_input
        .iter()
        .map(|a| {
          a.value().map(|v| {
            // Hope there is a cleaner way to do this.
            // Byte implements Deref to get &u8.
            let value: &u8 = &v;
            let decoded = super::table::decode_char(*value as char);

```

```

        F::from(decoded as u64)
    })
    .collect();

    // Enable lookup selector in both rows.
    self.config.lookup_sel.enable(&mut region, 0)?;
    self.config.lookup_sel.enable(&mut region, 1)?;

    // Positions of the inputs as: (column, row)
    let positions = [(0, 0), (1, 0), (0, 1), (1, 1)];
    for (input, pos) in b64_input.iter().zip(positions.iter()) {
        // Need to transform into native first due to the restrictions on
        // the `copied()` method that is needed to transform `Value<&V>` ->
        Value<V>`.
        let input: AssignedNative<F> = input.clone().into();
        input.copy_advice(|| "Base64 char", &mut region, advice_cols[pos.0],
pos.1)?;
    }

    let mut result = vec![];
    for (output, pos) in decoded_outs.into_iter().zip(positions.iter()) {
        let assigned = region.assign_advice(
            || "Base64 decoded values",
            advice_cols[pos.0 + 2], // same position as inputs, just 2 columns
right
            pos.1,
            || output,
        )?;
        result.push(assigned);
    }

    Ok(result)
},
)?;
Ok(decoded.try_into().unwrap())
}

```

The comment at the top of the function states that:

```

/// Returns a vector of 4 [AssignedNative] with each symbol
/// values. These values are guaranteed to be in the range [0,
2^6).

```

However, the gate constraints does not enforce this. The gate constraints only ensure that:

- $\text{decoded}_0 \cdot 2^8 + \text{decoded}_1 \in [0, 2^{12})$
- $\text{decoded}_2 \cdot 2^8 + \text{decoded}_3 \in [0, 2^{12})$

For instance, a malicious prover might assign:

- $\text{decoded}_0 = -1$

- $\text{decoded}_1 = 2^8$

As the “decoding” of `AA`.

Note that the `AssignedByte` invariant ensures `col_1` and `col_2` are within range $[0, 2^8)$, so no similar issue arises on the “inputs”.

Impact

The output of `base64_to_val_chunk` is only used by `val_to_ascii_chunk` which combines the limbs:

```
/// Receives 4 6-bit values, corresponding to a string of 4 base64
/// characters. Returns a vector of 3 [AssignedByte] with each symbol
/// values. These values are guaranteed to be in the range [0, 2^8).
fn val_to_ascii_chunk(
    &self,
    layouter: &mut impl Layouter<F>,
    b64_input: &[AssignedNative<F>; 4],
) -> Result<AsciiChunk<F>, Error> {
    // Sum ( b64_input[i] * 6^i ) = Sum ( byte[i] * 8^i)
    let terms = vec![
        (F::from(1u64 << (3 * 6)), b64_input[0].clone()),
        (F::from(1u64 << (2 * 6)), b64_input[1].clone()),
        (F::from(1u64 << 6), b64_input[2].clone()),
        (F::ONE, b64_input[3].clone()),
    ];
    let total = self
        .native_gadget
        .linear_combination(layouter, terms.as_slice(), F::ZERO)?;

    let bytes = self
        .native_gadget
        .assigned_to_be_bytes(layouter, &total, Some(3))?;

    Ok(bytes.try_into().unwrap())
}
```

The comment is inaccurate, the values will not be in the range $[0, 2^6)$, nevertheless the implementation is correct, since:

$$\text{b64_input}_0 \cdot 2^{3 \cdot 6} + \text{b64_input}_1 \cdot 2^{2 \cdot 6} + \text{b64_input}_2 \cdot 2^6 + \text{b64_input}_3$$

Can be written as:

$$(\text{b64_input}_0 \cdot 2^6 + \text{b64_input}_1) \cdot 2^{12} + (\text{b64_input}_2 \cdot 2^6 + \text{b64_input}_3)$$

And the `Base64 lookup` gate ensures that the two green/blue quantities are computed correctly – even though the individual components may not.

Hence, the *overall implementation is sound*, but could be optimized by eliminating the intermediate 6-bit value columns and using 12-bit values directly, reducing both the number of advice columns needed and avoiding the potential for out-of-range values in the intermediate steps. This solution requires no additional range constraints.

Client Response

Midnight optimized the implementation as recommended above, by having the `base64_to_val_chunk` function return two 12-bit limbs instead for the 4 ASCII characters. These two 12-bit limbs are combined into a single 24-bit value, which is then split into three 8-bit bytes inside the `val_to_ascii_chunk` function.

```
/// Receives 4 ascii characters as [AssignedByte] representing a base64
/// string. Returns a 2 [AssignedNative] with the value of each pair.
/// These values are guaranteed to be in the range [0, 2^12).
fn base64_to_val_chunk(
    &self,
    layouter: &mut impl Layouter<F>,
    b64_input: &B64Chunk<F>,
) -> Result<[AssignedNative<F>; 2], Error> {
    let advice_cols = self.config.advice_cols;
    // |-----|-----|-----|
    // | A      | B      | 0x0001 |
    // |-----|-----|-----|
    // | C      | D      | 0x0203 |
    // |-----|-----|-----|
    let decoded = layouter.assign_region(
        || "Base64 chunk",
        |mut region| {
            let decoded_outs: Vec<Value<F>> = b64_input
                .chunks_exact(2)
                .map(|vs| {
                    vs[0].value().zip(vs[1].value()).map(|(c0, c1)| {
                        let v0 = super::table::decode_char(c0 as char) as u64;
                        let v1 = super::table::decode_char(c1 as char) as u64;
                        F::from(v0 * (1 << 6) + v1)
                    })
                })
                .collect();

            // Enable lookup selector in both rows.
            self.config.lookup_sel.enable(&mut region, 0)?;
            self.config.lookup_sel.enable(&mut region, 1)?;

            // Positions of the inputs as: (column, row)
            let positions = [
                [(0, 0), (1, 0)], //
                [(0, 1), (1, 1)],
            ];
            for (input, pos) in b64_input.iter().zip(positions.as_flattened()) {
                let input: AssignedNative<F> = input.clone().into();
                input.copy_advice(|| "Base64 char", &mut region, advice_cols[pos.0],
pos.1)?;
            }
        }
    )
}
```

```

        let result: Result<Vec<_, _> = decoded_outs
            .into_iter()
            .zip(positions)
            .map(|(output, pos)| {
                region.assign_advice(
                    || "Base64 decoded values",
                    advice_cols[pos[0].0 + 2], /* same position as inputs, just 2
                                                * columns right */
                    pos[0].1,
                    || output,
                )
            })
            .collect();
        result
    },
    )?;

    Ok(decoded.try_into().unwrap())
}

/// Receives 2 12-bit values, where each value represents a pair base64
/// characters. Returns a vector of 3 [AssignedByte] with each symbol
/// values. These values are guaranteed to be in the range [0, 2^8).
fn val_to_ascii_chunk(
    &self,
    layouter: &mut impl Layouter<F>,
    b64_input: &[AssignedNative<F>; 2],
) -> Result<AsciiChunk<F>, Error> {
    let terms = vec![
        (F::from(1u64 << 12), b64_input[0].clone()),
        (F::ONE, b64_input[1].clone()),
    ];
    let total = self
        .native_gadget
        .linear_combination(layouter, terms.as_slice(), F::ZERO)?;

    let bytes = self
        .native_gadget
        .assigned_to_be_bytes(layouter, &total, Some(3))?;

    Ok(bytes.try_into().unwrap())
}

```

#02 - Batch Verification subject to Truncation

Severity: Low **Location:** compact_std_lib.rs

Description

The `zkstdLib` provides a way to verify batch of proofs jointly using the `batch_verify` function. This takes a slice of verification keys `vks`, public inputs `pis`, and proofs `proofs`. The intended effect is that all these arrays have the same length and there is a number of debug assertions checking this:

```
/// Verifies a batch of proofs with respect to their corresponding vk.
/// This method does not need to know the `Relation` the proofs are associated
/// to and, indeed, it can verify proofs from different `Relation`s.
/// For that, this function does not take `instance`s, but public inputs
/// in raw format (`Vec<F>`).
///
/// Returns `Ok(())` if all proofs are valid.
pub fn batch_verify(
    params_verifier: &[ParamsVerifierKZG<blstrs::Bls12>],
    vks: &[MidnightVK],
    pis: &[Vec<F>],
    proofs: &[Vec<u8>],
) -> Result<(), Error> {
    let n = params_verifier.len();
    debug_assert_eq!(vks.len(), n);
    debug_assert_eq!(pis.len(), n);
    debug_assert_eq!(proofs.len(), n);

    let guards = vks
        .iter()
        .zip(pis.iter())
        .zip(proofs.iter())
        .map(|((vk, pi), proof)| {
            ...
        })
        .collect::<Result<Vec<_>, Error>>()?;

    if DualMSM::batch_verify(guards.into_iter(), params_verifier.iter()).is_ok() {
        Ok(())
    } else {
        Err(Error::Synthesis)
    }
}
```

The issue is when the slices do not have the same length: in debug mode this causes a crash (a potential availability issue) however in release mode the effect is on soundness. If the slices are not the same length, the `zip` ing of the iterators:

```
let guards = vks
    .iter()
```

```
.zip(pis.iter())
.zip(proofs.iter())
```

Will simply truncate the iterator to the shortest one, hence in a scenario where a malicious prover can invoke this method with e.g. the empty slice for the proofs, this will cause the verifier to accept any set of `vks` and public inputs `pis`. For instance, one could imagine a scenario where the verifier simply deserialized a `Vec<Vec<u8>>` from the network and treats the joint vector as a proof for a sequence of `vks` and `pis` collected by the verifier himself.

We suggest adding a check that the lengths of each slice is the same and return an error if they differ, something like:

```
/// Verifies a batch of proofs with respect to their corresponding vk.
/// This method does not need to know the `Relation` the proofs are associated
/// to and, indeed, it can verify proofs from different `Relation`s.
/// For that, this function does not take `instance`s, but public inputs
/// in raw format (`Vec<F>`).
///
/// Returns `Ok(())` if all proofs are valid.
pub fn batch_verify(
    params_verifier: &[ParamsVerifierKZG<blstrs::Bls12>],
    vks: &[MidnightVK],
    pis: &[Vec<F>],
    proofs: &[Vec<u8>],
) -> Result<(), Error> {
    let n = params_verifier.len();
    if vks.len() != n || pis.len() != n || proofs.len() != n {
        return Err(Error::InvalidInput);
    }
    ...
}
```

Impact

If the caller fails to compare the lengths of the slices, a malicious prover can cause a crash or potentially a soundness issue.

Client Response

Midnight added an *explicit* length check to ensure that the lengths of the slices are the same:

```
pub fn batch_verify(
    params_verifier: &[ParamsVerifierKZG<blstrs::Bls12>],
    vks: &[MidnightVK],
    pis: &[Vec<F>],
    proofs: &[Vec<u8>],
) -> Result<(), Error> {
    let n = params_verifier.len();
    if vks.len() != n || pis.len() != n || proofs.len() != n {
        // TODO: have richer types in halo2
    }
}
```

```
        return Err(Error::InvalidInstances);  
    }  
    ...  
}
```

#03 - State Management and Implicit Assumptions in Poseidon Implementation

Severity: Low Location: poseidon

Description

We observed several issues related to state management and implicit assumptions within the Poseidon implementation:

- **Passing Inputs Instead of State to Full Round:** In the initial phase of Poseidon, the `state` passed to the first full round (line 413) originates from the region where inputs are assigned. However, later code expects `state` to belong to the permutation region. Although both usages share the same Rust type, they refer to different regions internally. We recommend explicitly passing the `inputs` array to the first full round and declaring a new `state` variable to avoid any hidden discrepancies.
- **Implicit Dependency on Column Sharing Between Chips:** In the first row of the full round, there is an implicit assumption that the correct `state` will already be present. This occurs because `native_gadget` and `poseidon_chip` **share the same advice columns** (see references [here](#) and [here](#) for `native_gadget` and `poseidon_chip`, respectively). This setup currently works because the `add_constants_in_region` function writes to the same columns that the full round gate later reads, with the correct offset enforced. However, this reliance on shared columns is not explicitly checked or documented.
- **Potential Impact of Changing Logic:** Although this dependency is not exploitable at the moment, if the logic surrounding column usage changes in the future, it could silently break the soundness of the Poseidon implementation.

Impact

While the current behavior does not introduce a security vulnerability, it creates a fragile dependency that could lead to soundness bugs if future modifications are made without awareness of these implicit assumptions.

Recommendation

We recommend taking one or more of the following actions:

- Explicitly **assert** that `native_gadget` and `poseidon_chip` use the same advice columns.
- **Document** the implicit assumption clearly with special comments in the code, making it easier to detect and audit related changes in the future.

Client Response

An explicit assert was added which checks that the state cells passed are indeed in the correct columns:

```
// The first full round assumes that `add_constants_in_region` above assigns  
// `state` in the same columns as Poseidon in the region. This is checked by the  
// below assertion.  
assert!(state  
    .iter()  
    .zip(self.config.register_cols)  
    .all(|(acell, col)| {  
        let col1: Column<Advice> = acell.cell().column.try_into().unwrap();  
        col1 == col  
    })  
);
```

This ensures that the native gadget and the Poseidon chip overlaps as expected.

#04 - Miscellaneous Remarks on JSON

Severity: Informational **Location:** json

Description

The JSON codebase contains several minor issues, including typos, inaccurate comments, redundant code, and unnecessarily complicated constructs. These issues are detailed below:

- **Optimize `get_subsequence`** : In `parser_gadget`, the `get_subsequence` function currently has a cost of $O(n \times len)$. By leveraging a `lookup_any` approach, the cost could be reduced to $O(n + len)$ by loading the string characters into a fixed/advice column pair:

IDX (FIXED)	CHAR (ADVICE)
0	x0
1	x1
...	...
n-1	x{n-1}

Subsequent substring extractions would then cost only $O(len)$ after the initial loading. This method would also provide an automatic range check on the index.

- **Clarify Naming in Base64 Decoder (`val_to_ascii_chunk`)**: In the base64 decoder:

```
fn val_to_ascii_chunk(  
    &self,  
    layouter: &mut impl Layouter<F>,  
    b64_input: &[AssignedNative<F>; 4],  
) -> Result<AsciiChunk<F>, Error>
```

The name `AsciiChunk` is misleading. Although it references ASCII, the method actually decodes base64 input to **bytes**, not ASCII characters. We recommend renaming the method and the associated struct to something like `base64_chunk_to_byte_chunk` and `ByteChunk`, respectively, for better clarity.

- **Unexpected Padding Behavior in Base64 Decoder**: The base64 decoder handles padding (`pad = true`) in an unexpected way. For example, `"QUEA"` and `"QUE="` are both decoded into `\x41\x41\x00`, even though the correct decoding of `"QUE="` would be `\x41\x41`. This behavior occurs because `"="` is replaced by `"A"` (0) before processing the 4-character chunks.

Additionally, strings like "QQ=A" are accepted and decoded, which may not be desirable depending on the broader use case. We recommend clarifying whether this behavior is intentional and acceptable in the intended context.

- **Insufficient Validation in `url_to_standard`:** The method:

```
fn url_to_standard(
    &self,
    layouter: &mut impl Layouter<F>,
    b64url_input: &[AssignedByte<F>],
) -> Result<Vec<AssignedByte<F>>, Error>
```

does not strictly validate the input as correctly base64url-encoded. If the input string already contains `_` or `/`, these are simply ignored during parsing. This relaxed behavior could lead to accepting improperly encoded inputs. We recommend making input validation stricter if conformance to standard base64url encoding is required.

Impact

These issues have no security implications if all inputs are from trusted sources (e.g., `url_to_standard` and padding logic). However, addressing them would significantly improve the codebase's readability, maintainability, and overall quality.

Recommendation

We recommend addressing the identified issues to enhance clarity, consistency, and maintainability.

Client Response

The client clarified that the base64 encoded string is assumed to be valid and that correctness is not ensured on malformed inputs: the wider application of the code is JSON web tokens, in which the base64 encoded payload is signed by a trusted authority. Hence most of these observations are not applicable in the wider application of the code.

#05 - Miscellaneous Remarks on Poseidon

Severity: Informational **Location:** poseidon

Description

The Poseidon codebase contains several minor issues, including typos, inaccurate comments, redundant code, and unnecessarily complicated constructs. These issues are detailed below:

- **Dead Code (Pallas and Vesta Constants):** The Pallas (`src/hash/poseidon/constants/pallas.rs`) and Vesta (`src/hash/poseidon/constants/vesta.rs`) constants are unused across the Midnight library. We recommend removing them.
- **Rename `round` Variable for Batched Partial Rounds:** The `round` variable actually represents a batch of partial rounds. We recommend renaming the `round` variable for full rounds to `full_round_index` and for partial batches to `partial_round_batch_index`. Similar renamings should be applied throughout the codebase where analogous variables appear.
- **Variable Shadowing in `partial_round_gate`:** The `ids` variable is shadowed on both lines [232](#) and [242](#). We recommend introducing distinct variable names for clarity.
- **Reduce Mutability for Better Code Clarity:** In some cases, the code could return results directly and rely on LLVM's copy elision, improving both readability and intent. Arrays can be mapped cleanly without unnecessary mutation. For example, [the following code](#):

```
fn linear_layer<F: PoseidonField>(  
    inputs: &[Expression<F>],           // Length WIDTH  
    outputs: &[Expression<F>],          // Length WIDTH  
    constants: &mut [Expression<F>],    // Length WIDTH  
) {  
    #[allow(clippy::needless_range_loop)]  
    for i in 0..WIDTH {  
        constants[i] = constants[i].clone() - outputs[i].clone();  
        for j in 0..WIDTH {  
            constants[i] =  
                constants[i].clone() + Expression::Constant(F::MDS[i][j]) *  
                inputs[j].clone();  
        }  
    }  
}
```

can be refactored to:

```
fn linear_layer<F: PoseidonField>(  
    inputs: [Expression<F>; WIDTH],
```

```

    outputs: [Expression<F>; WIDTH],
    constants: [Expression<F>; WIDTH],
) -> [Expression<F>; WIDTH] {
    let mut ids = constants;
    for i in 0..WIDTH {
        ids[i] = ids[i].clone() - outputs[i].clone();
        for j in 0..WIDTH {
            ids[i] = ids[i].clone() + Expression::Constant(F::MDS[i][j]) *
inputs[j].clone();
        }
    }
    ids
}

```

- **Introduce a Dedicated S-Box Constraint Function:** We recommend introducing a dedicated helper function for S-box constraints, which can then be used across the codebase (e.g., in `full_round_gate`):

```

fn sbox<F: PoseidonField>(x: Expression<F>) -> Expression<F> {
    x.clone() * x.square().square()
}

```

- **Simplify the `init` Function:** The following initialization logic:

```

fn init(nb_skips: usize) -> Self {
    RoundId {
        nb_skips,
        ids: core::array::from_fn(|i| {
            if i < WIDTH + 1 + nb_skips {
                let mut id = RoundVarId::init();
                if i < WIDTH {
                    id.var_coeffs[i] = F::ONE
                };
                id
            } else {
                RoundVarId::init()
            }
        }),
    }
}

```

can be simplified to:

```

fn init(nb_skips: usize) -> Self {
    RoundId {
        nb_skips,
        ids: core::array::from_fn(|i| {
            if i < WIDTH {
                let mut id = RoundVarId::init();
                id.var_coeffs[i] = F::ONE;
                id
            } else {
                RoundVarId::init()
            }
        })
    }
}

```

```

    }},
  }
}

```

- **Use `split_at` to Partition Slices Cleanly:** In several places where slices are manually split, we recommend using `.split_at` to more clearly indicate partitioning. For example, the `to_expression` function:

```

fn to_expression(self, vars: &[Expression<F>]) -> Expression<F> {
    let expr = self.var_coeffs[..WIDTH - 1].iter().zip(vars).fold(
        Expression::Constant(F::ZERO),
        |accu, (coeff, x)| {
            if coeff.is_zero_vartime() {
                accu
            } else {
                accu + Expression::Constant(*coeff) * x.clone()
            }
        },
    );
    self.var_coeffs[WIDTH - 1..]
        .iter()
        .zip(&vars[WIDTH - 1..])
        .fold(expr, |accu, (coeff, x)| {
            if coeff.is_zero_vartime() {
                accu
            } else {
                accu + Expression::Constant(*coeff) * x.clone().square().square() *
x.clone()
            }
        })
}

```

- **Simplify Modulo Assertions with `assert_eq!`:** Instead of manually checking modulo conditions with `panic!`, we recommend using `assert_eq!`:

Original:

```

#[allow(clippy::modulo_one)]
if NB_PARTIAL_ROUNDS % (1 + NB_SKIPS_CIRCUIT) != 0 {
    panic!(
        "The Poseidon chip assumes that the number of partial rounds (NB_PARTIAL_ROUNDS =
{}) is divisible by the number of round skips (1 + NB_SKIPS = {}).",
        NB_PARTIAL_ROUNDS,
        1 + NB_SKIPS_CIRCUIT
    )
};

```

Improved:

```

assert_eq!(
    NB_PARTIAL_ROUNDS % (1 + NB_SKIPS_CIRCUIT),
    0,
    "The Poseidon chip assumes that the number of partial rounds (NB_PARTIAL_ROUNDS = {})

```

```
is divisible by the number of round skips (1 + NB_SKIPS = {}).",  
    NB_PARTIAL_ROUNDS,  
    1 + NB_SKIPS_CIRCUIT  
);
```

Impact

These issues have no security implications. However, addressing them would significantly improve the codebase's readability, maintainability, and overall quality.

Recommendation

We recommend addressing the identified issues to enhance clarity, consistency, and maintainability.

Client Response

All of these suggested changes were implemented by the client.

#06 - Poseidon Parameters Generated by a Deprecated Script

Severity: Informational **Location:** poseidon

Location: `src/hash/poseidon/constants/blstrs.rs`

Description

The parameters (i.e., the round constants and the MDS matrix) for the BLS scalar field were generated using the [generate_parameters_grain.sage](#) script from the [pasta-hadeshash](#) repository. This repository is a fork of the [hadeshash repository](#) that adds support for the Pasta curves. However, the fork relies on a deprecated version of the Hadeshash repository.

Impact

The use of parameters generated by a deprecated script may expose the implementation to algebraic attacks described in this [paper](#).

Recommendation

We recommend using the latest version of the script to regenerate the parameters, ensuring improved security and robustness. Specifically, the script can be found [here](#), and the command to generate the required parameters is:

```
sage generate_params_poseidon.sage 1 0 255 3 5 136
0x73eda753299d7d483339d80809a1d80553bda402fffe5bfefffffffff00000001
Params: n=255, t=3, alpha=5, M=136, R_F=8, R_P=60
Modulus = 52435875175126190479447740508185965837690552500527637822603658699938581184513
Number of S-boxes: 84
Number of round constants: 204
...
```

Client Response

The client verified that the current parameters *already coincide* with those produced by the new script.

#07 - Miscellaneous Remarks on ZkStdLib

Severity: Informational **Location:** compact_std_lib

Location: src/compact_std_lib

Description

The ZkStdLib codebase contains several minor issues, including suboptimal type structure, potential logic issues, and unnecessarily complicated constructs. These issues are detailed below:

Review Comments

Improve Mutual Exclusivity Representation

In the `zkStdLib` struct, the fields `sha256_table11_chip` and `sha256_table16_chip` are mutually exclusive:

```
pub struct ZkStdLib {  
    // ...  
    sha256_table11_chip: Option<Table11Chip<F>>,  
    sha256_table16_chip: Option<Table16Chip<F>>,  
}
```

This could be better captured with an enum:

```
enum Sha256Table<F: Field> {  
    Table11(Table11Chip<F>),  
    Table16(Table16Chip<F>),  
    None  
}  
  
pub struct ZkStdLib {  
    // ...  
    sha256_table: Sha256Table<F>,  
}
```

Magic Constants

There are various magic constants in the code that would benefit from named constants or enums:

```
if arch.sha256.is_some() { 9 } else { 0 },
```

Implement Trait Instead of Custom Conversions

Instead of custom conversion methods:

```
impl ZkStdLibArch {
    fn to_byte(self) -> u8 {
        // ...
    }

    fn from_byte(byte: u8) -> Self {
        // ...
    }
}
```

Consider implementing `Into<u8>` and `From<u8>` / `TryFrom<u8>` to leverage Rust's standard conversion traits.

Version Tag Missing in Serialization

The serialization of `zkStdLibArch` lacks a version tag:

```
/// Writes the ZKStd architecture to a buffer.
pub fn write<W: io::Write>(&self, writer: &mut W) -> io::Result<()> {
    writer.write_all(&[self.to_byte()])
}

/// Reads the ZkStd architecture from a buffer.
pub fn read<R: io::Read>(reader: &mut R) -> io::Result<Self> {
    let mut byte = [0u8; 1];
    reader.read_exact(&mut byte)?;
    Ok(Self::from_byte(byte[0]))
}
```

Adding a version identifier would future-proof the format against changes in the meaning of bitfields.

Use `.expect()` Instead of `.unwrap_or_else()`

Instead of:

```
/// Native EccChip.
pub fn jubjub(&self) -> &EccChip<C> {
    self.jubjub_chip
        .as_ref()
        .unwrap_or_else(|| panic!("ZkStdArch must enable jubjub"))
}
```

The code would be clearer with:

```
pub fn jubjub(&self) -> &EccChip<C> {
    self.jubjub_chip.as_ref().expect("ZkStdArch must enable jubjub")
}
```

Impact

These issues have no immediate security implications, but addressing them would improve code quality, safety, and maintainability.

Recommendation

We recommend addressing the identified issues to enhance the robustness, clarity, and maintainability of the codebase.

Client Response

The client improved the logic which computes the number of required advice columns by removing the magic constants. They also added a version tag, addressed the small comment about `fn jubjub` and opted to not implement the `Sha256Table` enum.